

DON'T WASTE JUST DONATE

NAME: - KOLA NAVEEN

ROLE:-BACKEND DEVELOPER

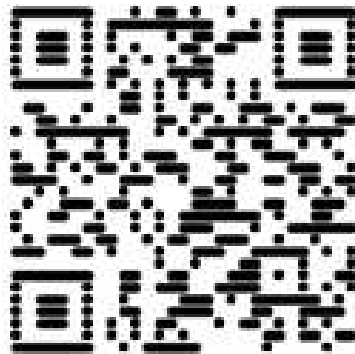
TEAM MEMBERS

B. DINESH REDDY

M. AJAY KUMAR REDDY

P. GNANESHWAR REDDY

Deployment Status (Live) :- <https://dwjd.vercel.app/>



ABSTRACT

Food wastage is a critical social issue that exists alongside widespread food insecurity. Large quantities of surplus food from households, events, and organizations are often discarded due to the absence of an efficient and reliable redistribution system. To address this problem, this project presents **DWJD (Don't Waste, Just Donate)**, a web-based donation and delivery management platform designed to connect donors, delivery riders, and recipients through a structured and secure workflow.

The system enables users to donate food and essential items by submitting donation details through an intuitive interface. Delivery riders can view available pickup requests, navigate to donor locations, verify donations using OTP-based authentication, and complete deliveries with real-time status tracking. An OTP verification mechanism ensures that donations are handed over securely and prevents unauthorized pickups or deliveries.

The platform also includes an **Admin module** that provides overall system insights such as user statistics, delivery performance, rejected pickups, and active promotional advertisements. This module allows administrators to monitor system activity and manage promotional content displayed to users, supporting scalability and future monetization.

The application is developed using **React and Vite** for the frontend, **Node.js and Express.js** for the backend, and **MongoDB** for data storage. Secure authentication is implemented using **JWT**, and email-based OTP verification is used to maintain trust and integrity throughout the donation lifecycle. The system is deployed on cloud platforms to ensure accessibility, performance, and reliability.

Overall, DWJD provides a practical, scalable, and technology-driven solution to reduce food wastage while promoting community-driven social responsibility.

INTRODUCTION

Food wastage has become a serious global challenge despite the growing number of people facing food insecurity. Large quantities of edible food are discarded daily from households, restaurants, events, and organizations due to the lack of a proper redistribution mechanism. At the same time, NGOs, shelters, and individuals struggle to access timely food resources. This imbalance highlights the need for a structured and technology-driven solution that can bridge the gap between surplus food providers and those in need.

With the rapid advancement of web technologies, digital platforms can play a crucial role in addressing real-world social problems. However, many existing donation systems lack transparency, real-time tracking, verification mechanisms, and proper role-based management. Issues such as unauthorized pickups, lack of accountability, and poor coordination between donors and delivery personnel reduce trust and efficiency in traditional donation workflows.

To overcome these challenges, **DWJD (Don't Waste, Just Donate)** is developed as a full-stack web application that facilitates secure, transparent, and organized donation and delivery management. The platform connects three primary stakeholders: **Users (donors)**, **Riders (delivery personnel)**, and **Administrators**. Each role is provided with a dedicated interface and controlled access to ensure smooth operation and accountability.

The system introduces OTP-based verification for donation pickup and delivery, ensuring that items are handed over only to authorized riders. Role-based dashboards allow users to track donation status, riders to manage pickups and deliveries, and administrators to monitor system performance through aggregated

statistics and promotional management. By leveraging modern web technologies and cloud deployment, the platform ensures scalability, reliability, and accessibility across devices.

DWJD aims to reduce food wastage, improve donation transparency, and encourage community participation through a secure and user-friendly digital solution

PROBLEM STATEMENT

Food wastage remains a persistent issue in society, even though many individuals and organizations are willing to donate surplus food, clothes, and essential items. The absence of a centralized, reliable, and secure platform to manage donations results in inefficient distribution, lack of accountability, and misuse of donated resources. Existing informal donation methods rely heavily on manual coordination, which often leads to delayed pickups, incorrect handovers, and limited tracking of donated items.

One of the major challenges in donation systems is the **lack of verification and role-based control**. Without proper authentication mechanisms, it becomes difficult to ensure that donated items are collected by the correct delivery personnel and delivered to the intended destination. This can lead to unauthorized pickups, lost donations, or disputes between donors and volunteers. Additionally, donors usually have no visibility into whether their donations are successfully delivered, reducing trust in the system.

Another significant issue is the **absence of real-time monitoring and administrative oversight**. Without an admin-level view, it is hard to analyze overall system performance, such as the number of active users, riders, completed deliveries, rejected pickups, and ongoing promotions. This limits scalability and prevents organizations from improving donation workflows based on data-driven insights.

Furthermore, current systems rarely support **secure communication and validation mechanisms** like OTP verification, which are essential for ensuring delivery authenticity. The lack of structured dashboards for different stakeholders—users, riders, and administrators—creates confusion and reduces operational efficiency.

Therefore, there is a strong need for a **secure, role-based, and scalable donation management platform** that ensures transparency, verification, and efficient coordination between donors, delivery personnel, and administrators, while providing real-time tracking and analytical insights to reduce food wastage and improve community support systems.

PROPOSED SYSTEM

The proposed system is a **web-based donation management platform** designed to securely connect donors, delivery personnel (riders), and administrators through a role-based architecture. The system aims to minimize food and resource wastage by ensuring verified pickups, transparent delivery tracking, and centralized administrative monitoring.

Role-Based Architecture

The system is divided into three clearly defined roles:

- **User (Donor)** – Initiates donation requests.
- **Rider (Delivery Partner)** – Collects and delivers donations.
- **Admin** – Monitors overall system activity and manages promotions.

Each role is provided with a dedicated user interface and access permissions, ensuring security and operational clarity.

Secure Authentication and Verification

- Users and riders register using email and password credentials.
- Email-based **OTP verification** is used during signup and donation confirmation to ensure authenticity.
- Riders can pick up donations only after successful OTP verification, preventing unauthorized collection.
- Role validation ensures users cannot access rider or admin functionalities.

Donation Workflow Management

- Users can donate food, clothes, or other items by submitting donation details.
- Each donation generates a unique OTP shared with the donor.
- Riders can view available donation requests and lock a pickup.
- OTP verification is mandatory before proceeding with delivery.

- Donation status transitions through defined stages:
Not Picked → Picked → Delivered

Rider Delivery Management

- Riders can view nearby and available pickup requests.
- Pickups are locked to a single rider to prevent conflicts.
- Riders can track their assigned deliveries under the “My Rides” section.
- Successful delivery updates the donation status for both donor and rider views.

Administrative Control and Analytics

- Admin users access a dedicated dashboard without public exposure.
- The admin panel provides real-time statistics such as:
 - Total users and riders
 - Completed deliveries
 - Rejected pickups
 - Active advertisements
- Admins can create and manage promotional advertisements displayed to users after login.

Advertisement and Promotion Module

- A separate advertisement module allows admin-added promotions.
- Advertisements are stored in a dedicated database collection.
- Active promotions are dynamically displayed to users within the application.

Scalable and Secure Backend

- RESTful APIs manage authentication, donations, pickups, deliveries, and advertisements.
- MongoDB ensures structured and scalable data storage.
- JWT-based authentication secures user sessions.
- The system is deployed using cloud platforms for high availability.

User-Friendly Interface

- Modern UI built with React ensures smooth navigation.
 - Dynamic routing enables seamless role-based transitions.
 - Clean dashboards improve usability for all stakeholders.
-

Key Advantages of the Proposed System

- Eliminates unauthorized pickups through OTP verification
- Ensures transparency in donation tracking
- Provides role-specific dashboards
- Enables admin-level monitoring and promotions
- Reduces food and resource wastage efficiently

OBJECTIVES

The primary objective of the proposed system is to develop a **secure, reliable, and efficient donation management platform** that minimizes resource wastage by enabling verified collection and delivery of donated items. The system focuses on transparency, accountability, and ease of use for all stakeholders.

Main Objectives

1. To Reduce Food and Resource Wastage

Provide a digital platform that enables surplus food and usable items to reach people in need instead of being discarded.

2. To Ensure Secure and Verified Donations

Implement OTP-based verification mechanisms to ensure that donations are collected only by authorized riders.

3. To Enable Role-Based Access Control

Design separate user interfaces and permissions for donors, riders, and administrators to maintain system security and clarity.

4. To Streamline Donation Pickup and Delivery

Facilitate an organized workflow from donation creation to successful delivery with real-time status updates.

5. To Prevent Unauthorized Pickups

Ensure that riders cannot proceed with delivery or mark donations as completed without successful OTP verification.

6. To Provide Transparency in Donation Tracking

Allow users and riders to track donation status at every stage of the process.

7. To Support Administrative Monitoring

Enable administrators to view overall system statistics such as total users, riders, deliveries, and rejected pickups.

8. To Enable Promotion and Awareness

Provide an advertisement module for promoting partner organizations or initiatives through the platform.

9. To Improve User Experience

Develop a responsive and intuitive user interface for smooth interaction across devices.

10. To Build a Scalable System

Design the system using modern web technologies that support future enhancements and increased user load.

SCOPE OF THE PROJECT

The scope of this project is to design and implement a **web-based donation management system** that facilitates the collection, verification, and delivery of donated items through a structured and secure workflow. The system is intended to serve as a **centralized platform** connecting donors, riders, and administrators.

In-Scope Features

1. **User Registration and Authentication**
 - Secure signup and login for users and riders.
 - Email-based OTP verification to ensure authenticity.
2. **Donation Management**
 - Users can donate food, clothes, and other essential items.
 - Donations are recorded with complete details including quantity and pickup location.
3. **Rider Pickup and Delivery System**
 - Riders can view available donation requests.
 - Pickup is locked to a rider once selected.
 - OTP verification is mandatory before proceeding to delivery.
4. **Real-Time Donation Status Tracking**
 - Donation status updates from *not picked* → *picked* → *delivered*.
 - Visibility of status for both donors and riders.
5. **Admin Monitoring and Control**
 - Administrative dashboard displaying overall system statistics.
 - Monitoring of users, riders, completed deliveries, and rejected pickups.
6. **Advertisement and Promotion Module**
 - Admins can add and manage promotional content.
 - Advertisements are displayed to users within the application interface.
7. **Role-Based User Interface**
 - Separate UI and functionalities for users, riders, and administrators.
 - Access control based on authenticated roles.
8. **Data Security and Integrity**
 - Secure storage of user credentials using encryption.
 - Controlled access to sensitive operations such as delivery confirmation.

Out-of-Scope Features

- Online payment gateway integration.
 - Real-time GPS tracking of riders.
 - Mobile application version (Android/iOS).
 - Integration with third-party delivery services.
-

Future Scope

- Integration of live location tracking.
- Analytics-based impact reporting.
- AI-based donation demand prediction.
- Multi-language support for wider accessibility.

SYSTEM ARCHITECTURE

The system architecture of the **DWJD (Don't Waste, Just Donate)** platform follows a **three-tier architecture**, ensuring modularity, scalability, and maintainability. The application separates concerns into distinct layers for user interaction, business logic, and data management.

1. Presentation Layer (Frontend)

The presentation layer is responsible for all user interactions and user interface rendering.

Technologies Used:

- React.js (Vite)
- HTML5, CSS3
- JavaScript

Responsibilities:

- Provides role-based user interfaces for:
 - Users (Donors)
 - Riders (Delivery personnel)
 - Admin
 - Handles user input such as:
 - Signup and login forms
 - Donation submission
 - OTP verification
 - Displays real-time donation status updates
 - Communicates with the backend via RESTful APIs using HTTP requests
-

2. Application Layer (Backend / Server)

The application layer handles business logic and acts as a bridge between the frontend and database.

Technologies Used:

- Node.js
- Express.js

Responsibilities:

- User authentication and authorization

- OTP generation and verification
 - Donation workflow management:
 - Donation creation
 - Pickup assignment
 - OTP-based verification
 - Delivery confirmation
 - Role-based access control (User, Rider, Admin)
 - Advertisement and promotion management
 - Admin analytics and statistics computation
 - Secure API handling and request validation
-

3. Data Layer (Database)

The data layer is responsible for persistent storage of application data.

Technology Used:

- MongoDB (NoSQL Database)
- Mongoose ORM

Responsibilities:

- Stores user profiles and credentials
 - Maintains donation records and status transitions
 - Stores OTP details securely with expiration
 - Stores advertisement and promotion data
 - Ensures data consistency and integrity
-

4. Communication Flow

- The frontend sends HTTP requests (GET/POST) to the backend APIs.
 - The backend processes requests, applies business logic, and interacts with the database.
 - Responses are sent back to the frontend in JSON format.
 - OTP verification acts as a security checkpoint before sensitive actions like pickup and delivery.
-

5. Security Architecture

- Passwords are encrypted using hashing techniques.
- OTPs are time-bound and invalidated after verification.

- Role-based access prevents unauthorized operations.
 - Secure authentication tokens (JWT) are used for session management.
-

6. Deployment Architecture

- **Frontend** hosted on Vercel
- **Backend** hosted on Render
- **Database** hosted on MongoDB Atlas

This cloud-based deployment ensures high availability and scalability.

TECHNOLOGY STACK

The **DWJD (Don't Waste, Just Donate)** platform is developed using a modern web technology stack that ensures scalability, security, and ease of maintenance. The selected technologies support efficient handling of real-time data, role-based workflows, and secure transactions.

1. Frontend Technologies

The frontend is responsible for user interaction and dynamic content rendering.

Technologies Used:

- **React.js (Vite)** – Component-based frontend framework
- **JavaScript (ES6+)** – Client-side scripting
- **HTML5** – Structure of web pages
- **CSS3** – Styling and responsive layout

Purpose:

- Builds dynamic and interactive user interfaces
- Implements role-based UI for User, Rider, and Admin
- Handles form submissions, OTP verification, and navigation
- Communicates with backend APIs using HTTP requests

2. Backend Technologies

The backend manages business logic, security, and system workflows.

Technologies Used:

- **Node.js** – Server-side JavaScript runtime
- **Express.js** – Web application framework for Node.js

Purpose:

- Handles RESTful API requests
- Manages authentication and authorization
- Controls donation lifecycle (donation, pickup, delivery)
- Implements OTP verification and status tracking
- Provides admin analytics and advertisement management

3. Database Technologies

The database stores and manages application data persistently.

Technologies Used:

- **MongoDB** – NoSQL document-oriented database
- **Mongoose** – Object Data Modeling (ODM) library

Purpose:

- Stores user, rider, and admin information
- Maintains donation records and status transitions
- Stores OTPs with expiry for verification
- Manages advertisement and promotion data

4. Security Technologies

Security is integrated at multiple levels of the application.

Technologies Used:

- **JWT (JSON Web Token)** – Secure authentication
- **bcrypt** – Password hashing
- **OTP-based verification** – Transaction validation

Purpose:

- Protects user credentials
- Prevents unauthorized access
- Ensures verified donation and delivery operations

5. API & Communication Technologies

Technologies Used:

- **RESTful APIs**
- **JSON** – Data exchange format

Purpose:

- Enables communication between frontend and backend
- Standardizes data transfer and response handling

6. Deployment & Hosting Technologies

Technologies Used:

- **Vercel** – Frontend hosting
- **Render** – Backend hosting
- **MongoDB Atlas** – Cloud database service

Purpose:

- Ensures high availability and scalability
- Simplifies continuous deployment
- Provides reliable cloud infrastructure

MODULE DESCRIPTION

The DWJD (Don't Waste, Just Donate) system is divided into three primary modules based on user roles:

1. **User Module**
2. **Rider Module**
3. **Admin Module**

Each module has clearly defined responsibilities and access permissions.

1. User Module

The **User Module** is designed for individuals or organizations who wish to donate surplus food or other usable items.

Functionalities

- User registration and login with OTP-based email verification
- Profile creation and profile management
- Donation creation with item details
- OTP generation for secure pickup confirmation
- Tracking donation status (Not Picked, Picked, Delivered)
- Viewing donation history and requests

Workflow

1. User registers and verifies email using OTP
2. User logs in securely
3. User submits donation details
4. System generates an OTP and stores it securely
5. User shares OTP with rider at pickup time
6. Donation status updates automatically after delivery

Purpose

- Enable safe and easy donation submission
- Ensure transparency in donation lifecycle
- Prevent misuse through OTP verification

2. Rider Module

The **Rider Module** is responsible for collecting donated items and delivering them to the appropriate destination.

Functionalities

- Rider registration with location access
- Rider login and authentication
- Viewing available pickup requests
- Locking a pickup request to prevent conflicts
- Navigating to pickup location
- OTP verification to confirm pickup
- Delivery confirmation and status update
- Viewing assigned and completed deliveries

Workflow

1. Rider logs in
2. Rider views available pickup requests
3. Rider locks a donation request
4. Rider navigates to pickup location
5. Rider verifies OTP provided by user
6. Donation is marked as picked
7. Rider delivers item and marks it as delivered

Purpose

- Ensure controlled and verified pickup process
- Avoid duplicate pickups
- Maintain accurate delivery tracking

3. Admin Module

The **Admin Module** provides administrative control and system-level monitoring.

Functionalities

- Secure admin login (predefined credentials)
- Viewing overall system statistics
- Monitoring total users, riders, and donations
- Tracking delivered and rejected donations
- Managing promotional advertisements

- Controlling advertisement visibility

Workflow

1. Admin logs in using predefined credentials
2. Admin accesses dashboard
3. Admin views real-time platform statistics
4. Admin creates and manages advertisements
5. Advertisements are displayed to users after login

Purpose

- Maintain system oversight
- Analyze platform usage
- Support sustainability through promotions

DATABASE DESIGN

The DWJD (Don't Waste, Just Donate) system uses a **MongoDB NoSQL database** to store and manage application data efficiently.

MongoDB is chosen due to its flexibility, scalability, and seamless integration with Node.js.

The database is structured into **multiple collections**, each representing a core entity in the system.

1. User Collection

Purpose

Stores all registered users including donors, riders, and administrators.

Collection Name

users

Fields Description

Field Name	Data Type	Description
_id	ObjectId	Unique identifier
username	String	Unique username
email	String	User email address
password	String	Encrypted password
first_name	String	User first name
last_name	String	User last name
phone	String	Contact number
profile_image	String	Profile picture URL
user_type	String	user, rider, or admin
rider_location	Object	Latitude & longitude (rider only)
is_verified	Boolean	Email verification status
is_active	Boolean	Account status
otp	String	OTP for verification
otp_expiry	Date	OTP expiration time
createdAt	Date	Account creation time
updatedAt	Date	Last update time

Notes

- Passwords are stored using **bcrypt hashing**
- OTP fields are cleared after successful verification
- Admin accounts are predefined

2. Donation Collection

Purpose

Stores all donation-related information from creation to delivery.

Collection Name

donations

Fields Description

Field Name	Data Type	Description
_id	ObjectId	Donation ID
donor_email	String	Donor's email
donor_name	String	Donor's name
item_type	String	Food / Clothes / Other
item_name	String	Item description
quantity	Number	Quantity of item
price_type	String	Free / Paid
price_amount	Number	Amount if paid
pickup_location	String	Pickup address
remarks	String	Additional notes
rider_email	String	Assigned rider
donation_status	String	Not Picked / Picked / Delivered
otp	String	Pickup OTP
otp_expiry	Date	OTP expiration
is_verified	Boolean	Pickup verification status
rejection_reason	String	Reason if rejected
picked_at	Date	Pickup timestamp
delivered_at	Date	Delivery timestamp
createdAt	Date	Donation created
updatedAt	Date	Last update

Notes

- OTP ensures secure pickup
- Status flow enforces delivery logic
- One rider can handle multiple donations

3. Advertisement Collection

Purpose

Stores promotional content displayed to users after login.

Collection Name

advertisements

Fields Description

Field Name	Data Type	Description
_id	ObjectId	Advertisement ID
title	String	Advertisement title
description	String	Promotional content
company_name	String	Sponsor company
amount_paid	Number	Promotion fee
is_active	Boolean	Visibility control
createdAt	Date	Created time
updatedAt	Date	Last update

Notes

- Only active ads are shown to users
- Admin controls creation and visibility

Database Relationships

Entity	Relationship
User → Donation	One-to-Many
Rider → Donation	One-to-Many
Admin → Advertisement	One-to-Many

Database Design Advantages

- Flexible schema for evolving requirements

- Fast read/write operations
- Secure password and OTP handling
- Scalable for large user base
- Easy integration with REST APIs

API DESIGN

The DWJD (Don't Waste, Just Donate) system follows a **RESTful API architecture** built using **Node.js and Express.js**.

The API layer acts as a bridge between the frontend (React) and the database (MongoDB), ensuring secure, structured, and scalable communication.

All APIs follow:

- Stateless request-response model
- JSON-based data exchange
- Role-based access (User, Rider, Admin)

1. Authentication & Authorization APIs

1.1 User Signup

Endpoint:

POST /signup

Purpose:

Registers a new user or rider and sends OTP for email verification.

Request Body:

```
{
  "username": "john_doe",
  "email": "john@gmail.com",
  "password": "*****",
  "confirm_password": "*****",
  "user_type": "user | rider",
  "latitude": "optional",
  "longitude": "optional"
}
```

Response:

```
{
  "status": "signup_success_otp_sent"
}
```

1.2 Verify OTP

Endpoint:

POST /verify-otp

Purpose:

Verifies the email using OTP.

Request Body:

```
{
  "email": "john@gmail.com",
  "otp": "123456"
}
```

Response:

```
{
  "status": "account_verified"
}
```

1.3 Login

Endpoint:

POST /login

Purpose:

Authenticates user, rider, or admin and returns JWT token.

Request Body:

```
{
  "email": "john@gmail.com",
  "password": "*****",
  "user_type": "user | rider"
}
```

Response:

```
{
  "status": "login_success",
  "token": "JWT_TOKEN",
  "user": {
    "email": "john@gmail.com",
    "user_type": "user"
  }
}
```

2. Donation APIs (User Side)

2.1 Create Donation

Endpoint:

POST /donate

Purpose:

Allows users to create donation requests.

Request Body:

```
{
  "donor_email": "user@gmail.com",
  "item_type": "food",
  "item_name": "Cooked Rice",
  "quantity": 5,
  "pickup_location": "Hyderabad"
}
```

Response:

```
{
  "status": "otp_sent",
  "donation_id": "ObjectId"
}
```

2.2 My Requests

Endpoint:

POST /my-requests

Purpose:

Displays all donations created by the logged-in user.

3. Rider APIs

3.1 View Available Pickups

Endpoint:

POST /rider/available-pickups

Purpose:

Lists all unpicked donation requests.

3.2 Lock Pickup

Endpoint:

POST /rider/pickup

Purpose:

Locks a donation so no other rider can pick it.

3.3 Verify Donation OTP

Endpoint:

POST /verify-donate-otp

Purpose:

Verifies donor OTP and confirms pickup.

3.4 Reject Pickup

Endpoint:

POST /rider/reject-pickup

Purpose:

Allows rider to reject pickup with a reason.

3.5 Mark Delivered

Endpoint:

POST /rider/mark-delivered

Purpose:

Marks the donation as successfully delivered.

3.6 My Rides

Endpoint:

POST /rider/my-rides

Purpose:

Displays all picked and delivered donations by rider.

4. Admin APIs

4.1 Admin Overview

Endpoint:

GET /admin/overview

Purpose:

Provides dashboard statistics.

Response:

```
{
  "totalUsers": 120,
  "totalRiders": 25,
  "delivered": 310,
  "rejected": 14,
  "activeAds": 5
}
```

4.2 Add Advertisement

Endpoint:

POST /admin/add-advertisement

Purpose:

Creates a new promotional advertisement.

4.3 Get Active Advertisements

Endpoint:

GET /advertisements

Purpose:

Fetches active promotions for users.

5. API Security Measures

- Password hashing using **bcrypt**
- JWT-based authentication
- OTP validation with expiry time
- Role-based route access
- Input validation and error handling

SECURITY IMPLEMENTATION

Security is a critical aspect of the DWJD (Don't Waste, Just Donate) platform, as it handles user authentication, personal data, and donation transactions. The system implements multiple layers of security to protect data confidentiality, integrity, and authorized access.

1. Authentication Security

1.1 Password Encryption

- User, Rider, and Admin passwords are **never stored in plain text**.
- Passwords are encrypted using **bcrypt hashing algorithm** before storing in the database.
- During login, the entered password is verified using secure hash comparison.

Benefit:

Prevents password leakage even if the database is compromised.

1.2 OTP-Based Verification

- OTP (One-Time Password) is generated during:
 - User registration
 - Donation confirmation
- OTP is:
 - 6-digit random number
 - Time-bound (expires after 10 minutes)
- OTP is stored temporarily and removed after successful verification.

Benefit:

Ensures only legitimate users and donors can perform sensitive actions.

2. Authorization & Role-Based Access Control

The system follows **Role-Based Access Control (RBAC)** with three roles:

Role	Access Level
User	Donate items, view requests
Rider	Pick, verify, and deliver donations
Admin	Monitor system statistics and manage promotions

- Routes are protected based on the user role.
- Admin access is restricted strictly to admin accounts.
- Riders cannot access user-only or admin-only endpoints.

Benefit:

Prevents unauthorized access and privilege escalation.

3. Token-Based Security (JWT)

- On successful login, a **JSON Web Token (JWT)** is issued.
- Token contains:
 - User ID
 - Role (user / rider / admin)
- Token has an expiry time (7 days).

Benefit:

Provides stateless and secure session management.

4. Data Validation & Input Sanitization

- All incoming API requests are validated:
 - Required fields are checked
 - Invalid or missing data is rejected
- Server-side validation prevents:
 - Invalid input
 - Malformed requests

Benefit:

Protects against injection attacks and malformed data submission.

5. OTP & Transaction Integrity

- Donation pickup requires **OTP verification from donor**.
- Without OTP verification:
 - Delivery cannot be marked as completed
- Prevents riders from falsely marking deliveries.

Benefit:

Ensures trust and transparency between users and riders.

6. Secure Database Design

- Sensitive fields such as:

- Passwords
 - OTP values
- Are either encrypted or removed after use.
- MongoDB schema enforces:
 - Field types
 - Required constraints

Benefit:

Maintains data consistency and reduces exposure of sensitive data.

7. CORS & API Protection

- CORS is enabled to allow only trusted frontend origins.
- Prevents unauthorized cross-origin requests.

Benefit:

Protects backend APIs from malicious external access.

8. Error Handling & Logging

- Generic error messages are returned to clients.
- Detailed error logs are maintained on the server.
- Prevents exposure of internal system details.

DEPLOYMENT DETAILS

The DWJD (Don't Waste, Just Donate) application is deployed using modern cloud platforms to ensure high availability, scalability, and ease of access. The system follows a **client-server deployment model**, where the frontend and backend are hosted independently.

1. Deployment Architecture Overview

The application is deployed in a **three-tier architecture**:

1. **Frontend Layer** – User Interface
2. **Backend Layer** – Business Logic & APIs
3. **Database Layer** – Persistent Data Storage

Each layer is deployed on a dedicated platform to improve performance and maintainability.

2. Frontend Deployment

Platform Used:

Vercel

Details:

- The frontend is developed using **React + Vite**.
- The production build is generated using Vite's optimized build process.
- The build output is deployed on **Vercel**, which provides:
 - Global CDN
 - Fast page loads
 - Automatic HTTPS
- Environment variables are configured to connect with the backend API hosted on Render.

Benefits:

- Fast global delivery
- Automatic redeployment on code updates
- Secure and scalable hosting

3. Backend Deployment

Platform Used:

Render

Details:

- Backend is built using **Node.js and Express.js**.
- RESTful APIs handle authentication, donations, rider operations, admin management, and advertisements.
- The backend is deployed as a web service on Render.
- Environment variables include:
 - MongoDB connection string
 - JWT secret
 - Email service credentials
- Health check endpoint is used to monitor service availability.

Benefits:

- Simple CI/CD pipeline
- Automatic server restarts
- Easy environment configuration

4. Database Deployment

Platform Used:

MongoDB Atlas (Cloud Database)

Details:

- MongoDB Atlas provides a fully managed cloud database.
- Collections include:
 - Users
 - Donations
 - Advertisements
- Secure connection using MongoDB URI.
- Database access is restricted using IP whitelisting and authentication credentials.

Benefits:

- High availability
- Automatic backups
- Scalable storage

5. Environment Configuration

Sensitive configuration values are stored using environment variables:

- MONGO_URI
- JWT_SECRET
- RESEND_API_KEY
- PORT

This ensures:

- Security of credentials
- Separation of development and production environments

6. Deployment Workflow

1. Code is developed and tested locally.
2. Changes are pushed to the Git repository.
3. Frontend:
 - Automatically deployed by Vercel on push.
4. Backend:
 - Automatically redeployed on Render.
5. Database remains persistent on MongoDB Atlas.

7. Security in Deployment

- HTTPS is enforced on both frontend and backend.
- JWT-based authentication protects API endpoints.
- Database credentials are never exposed in client-side code.
- CORS policies restrict unauthorized access.

8. Scalability & Reliability

- Frontend uses CDN-based delivery.
- Backend can be scaled vertically or horizontally.
- Database supports automatic scaling.
- Stateless backend design allows easy load balancing.

TESTING STRATEGY

Testing plays a crucial role in ensuring that the DWJD (Don't Waste, Just Donate) application functions correctly, securely, and reliably. A structured testing strategy was followed throughout the development lifecycle to validate individual components, system integration, and real-world usage scenarios.

1. Testing Objectives

The main objectives of testing in this project are:

- To verify that all functionalities work as intended
- To ensure data integrity and security
- To detect and fix bugs at early stages
- To validate role-based access for Users, Riders, and Admin
- To ensure smooth end-to-end donation and delivery flow

2. Types of Testing Performed

2.1 Unit Testing

Unit testing focuses on testing individual modules and functions independently.

Examples:

- OTP generation and validation logic
- Password hashing and comparison
- Input validation for signup and donation forms
- API response handling

Outcome:

Each unit performed as expected with valid and invalid inputs.

2.2 Integration Testing

Integration testing ensures that different modules interact correctly with each other.

Examples:

- Frontend communicating with backend APIs
- Backend connecting to MongoDB database
- OTP verification flow between email service and backend
- Rider pickup flow after OTP verification

Outcome:

Data flow between modules was verified and worked without conflicts.

2.3 Functional Testing

Functional testing validates system features against functional requirements.

Key Scenarios Tested:

- User registration and email verification
- Login for User, Rider, and Admin roles
- Donation creation and OTP verification
- Rider pickup, delivery, and rejection flows
- Admin dashboard access and statistics retrieval
- Advertisement creation and display

Outcome:

All functional requirements were met successfully.

2.4 Role-Based Testing

Role-specific access was tested thoroughly:

Role	Tested Features
User	Donate items, view donation status
Rider	View pickups, verify OTP, deliver items
Admin	View statistics, manage advertisements

Outcome:

Unauthorized access was blocked, and role isolation was maintained.

2.5 API Testing

API endpoints were tested using tools like **Postman**.

Tested Aspects:

- Request/response structure
- HTTP status handling
- Error responses for invalid data
- Authentication and authorization checks

Outcome:

All APIs returned correct responses and error messages.

2.6 Database Testing

Database operations were tested to ensure:

- Correct data insertion and retrieval
- Proper schema enforcement
- Relationship consistency between users, donations, and advertisements

Outcome:

Data persisted correctly without redundancy or corruption.

2.7 UI / UX Testing

User interface testing ensured:

- Responsive design across devices
- Proper navigation between pages
- Clear error and success messages
- Smooth user experience for all roles

Outcome:

The UI was intuitive and user-friendly.

3. Manual Testing Approach

Since the application is a real-time system with role-based workflows, **manual testing** was primarily used to simulate real-world scenarios such as:

- Network delays
- Interrupted OTP flows
- Rider abandoning pickup mid-process
- Multiple users interacting simultaneously

4. Error Handling & Debugging

- Console logs were used for backend debugging
- Browser developer tools were used for frontend debugging
- Graceful error messages were displayed to users
- Edge cases like expired OTPs and invalid roles were handled

5. Testing Environment

- **Frontend:** Localhost & Vercel (Production)
- **Backend:** Localhost & Render (Production)
- **Database:** MongoDB Atlas
- **Tools:** Browser DevTools, Postman

CHALLENGES FACED

During the development of the DWJD (Don't Waste, Just Donate) application, several technical and practical challenges were encountered. These challenges played a crucial role in improving system design, debugging skills, and real-world problem-solving abilities.

1. OTP Delivery Delays and Failures

Challenge

- OTP emails were delivered instantly in localhost but were delayed or blocked in production (Render + Gmail).
- Free email services imposed restrictions and spam filtering.

Solution

- Analyzed SMTP limitations and identified Gmail throttling.
- Integrated **Resend Email API** for faster and reliable OTP delivery.
- Implemented non-blocking OTP sending to avoid API timeouts.

Learning:

Production environments behave differently from local systems.

2. Role-Based Authentication Complexity

Challenge

- Handling three roles (User, Rider, Admin) with different access levels.
- Preventing unauthorized page access.
- Admin login needed to work without exposing admin options publicly.

Solution

- Implemented role detection from database instead of frontend selection.
- Restricted UI rendering based on `user_type`.
- Secured admin dashboard access through backend validation.

Learning:

Authentication must be enforced on both frontend and backend.

3. OTP-Based Pickup Verification Logic

Challenge

- Preventing riders from completing deliveries without OTP verification.
- Handling edge cases where the rider leaves the page mid-process.

Solution

- Introduced OTP verification state before enabling delivery.
- Ensured backend checks OTP verification before status changes.
- Added recovery logic to resume incomplete pickups safely.

Learning:

State management is critical in real-time workflows.

4. Navigation and Route Management Issues

Challenge

- Blank screens due to incorrect routing.
- Nested routes not rendering expected components.
- Admin routes conflicting with user routes.

Solution

- Reorganized routes using nested routing with `Outlet`.
- Fixed incorrect paths (`/overall` → `/afterlogin/overall`).
- Centralized route control inside `Afterlogin.jsx`.

Learning:

Proper route hierarchy prevents UI failures.

5. MongoDB Schema and Data Consistency

Challenge

- Missing fields caused runtime errors.
- Status mismatches between frontend and backend.

Solution

- Refined Mongoose schemas with defaults and enums.
- Enforced validation at both schema and API levels.
- Added timestamps for better tracking.

Learning:

A strong database schema reduces bugs significantly.

6. Deployment Environment Differences

Challenge

- APIs working locally but failing in production.
- Environment variables not loading correctly.

Solution

- Verified environment variables in Render dashboard.
- Added health-check APIs.
- Logged server behavior for production debugging.

Learning:

Deployment testing is as important as development testing.

7. Advertisement Module Integration

Challenge

- Integrating promotions dynamically into user UI.
- Ensuring only active advertisements are shown.

Solution

- Created a dedicated `advertisements` collection.
- Implemented admin-only advertisement creation.
- Filtered active promotions using backend APIs.

Learning:

Modular design simplifies feature expansion.

8. Error Handling and User Experience

Challenge

- Raw server errors confusing users.
- No feedback during failures.

Solution

- Added user-friendly error messages.
- Implemented loading states and alerts.
- Logged errors internally for debugging.

Learning:

Good UX includes clear error communication.

FUTURE ENHANCEMENTS

The DWJD (Don't Waste, Just Donate) platform has been successfully implemented with core functionalities. However, several enhancements can be introduced in the future to improve scalability, usability, and impact.

1. Real-Time Notifications

- Push notifications or SMS alerts for:
 - Pickup acceptance
 - OTP verification
 - Delivery completion
- Reduces dependency on email-based OTPs.

Benefit: Faster communication and improved user engagement.

2. Live Location Tracking

- Real-time tracking of rider movement during pickup and delivery.
- Display estimated arrival time to donors.

Benefit: Transparency and better coordination.

3. Advanced Admin Analytics Dashboard

- Graph-based analytics (daily, weekly, monthly).
- Donation success rates.
- Rider performance metrics.

Benefit: Data-driven decision-making.

4. Payment Gateway Integration

- Secure online payments for promoted advertisements.
- Automated invoice generation.

Benefit: Monetization and professional business workflow.

5. Mobile Application Support

- Native Android/iOS apps using React Native or Flutter.
- Offline support for riders.

Benefit: Wider reach and better accessibility.

6. Multi-Organization Support

- Allow NGOs, shelters, and food banks to register.
- Role-based access for organizations.

Benefit: Scalable ecosystem for donations.

7. Automated Rider Assignment

- Intelligent rider allocation based on distance and availability.
- Reduced manual selection.

Benefit: Faster pickups and optimized logistics.

8. AI-Based Demand Prediction

- Predict food demand based on location and time.
- Reduce wastage by prioritizing urgent donations.

Benefit: Smarter resource utilization.

9. Enhanced Security Features

- Two-factor authentication (2FA).
- IP-based login alerts.
- Token refresh mechanism.

Benefit: Stronger protection against unauthorized access.

10. Feedback & Rating System

- Ratings for riders and donors.
- Review-based trust score.

Benefit: Improves accountability and service quality.

CONCLUSION

The DWJD (Don't Waste, Just Donate) platform successfully addresses the problem of food and resource wastage by providing a secure, structured, and technology-driven solution. The system efficiently connects donors with riders to ensure timely pickup and delivery of donated items, thereby reducing waste and supporting people in need.

The project demonstrates the practical application of full-stack development concepts, including user authentication, role-based access control, OTP verification, RESTful APIs, database design, and real-time workflow management. The inclusion of separate modules for users, riders, and administrators ensures clarity, security, and scalability within the system.

By implementing an admin dashboard with analytics and promotional management, the platform also introduces a sustainable operational model. The system is designed with future expansion in mind, allowing easy integration of advanced features such as live tracking, mobile applications, and intelligent automation.

Overall, DWJD is a reliable, scalable, and socially impactful application that combines technical efficiency with real-world problem solving. This project reflects strong backend logic, clean frontend design, and thoughtful system architecture, making it suitable for real-time deployment and further enhancement.